



colabor-ai

Plataforma de Orquestracao de Agentes de IA Colaborativos

Documentacao completa da arquitetura, modulos, agentes e protocolo de comunicacao

Versao 2.1

Maio de 2026

github.com/adm4670/colabor-ai

Sumario

1. Visao Geral

1.1 O que e o colabor-ai?

1.2 Filosofia do Projeto

2. Arquitetura Geral

2.1 Diagrama de Alto Nivel

2.2 Fluxo de uma Conversa

3. Modulo Core (Engine)

3.1 Agent

3.2 AgentOrchestrator

3.3 ToolRegistry & Tools

3.4 Agentes Especializados

4. Modulo Cloud (Backend)

4.1 Servidor (Express + WebSocket)

4.2 Orchestrator Cloud Edition

4.3 Memory Engine

4.4 Context Engine

4.5 Autenticacao (JWT)

5. Modulo Client (Desktop)

5.1 Visao Geral do Client

5.2 Ferramentas Locais

6. Protocolo de Comunicacao

6.1 WebSocket Messages

6.2 Fluxo Tool Call/Result

7. Stack Tecnologica

8. Guia de Inicio Rapido

9. Roadmap

1. Visao Geral

1.1 O que e o colabor-ai?

O **colabor-ai** e uma plataforma open-source para criar e orquestrar **agentes de IA especializados** que colaboram entre si para resolver tarefas. Imagine uma equipe de mini-especialistas em IA, cada um excelente em uma area diferente, coordenados por um maestro inteligente.

Analogia: Assim como uma orquestra tem violinistas, pianistas e percussionistas, cada um tocando sua parte sob a regencia de um maestro, o colabor-ai tem agentes especializados (Chat, Python, FileSystem, etc.) orquestrados por um Planner que decide quem faz o que e em qual ordem.

Agente	Especialidade	Onde Roda
Planner	Decidir qual agente deve agir e com qual instrucao	Cloud
Assistant	Conversa geral, explicacoes, duvidas	Cloud
Python	Calculos, analise de dados, execucao de codigo	Cloud
Writer	Formatacao da resposta final para o usuario	Cloud
File System	Ler, criar, editar e organizar arquivos no PC	Client (Windows)
Shell	Executar comandos CMD e PowerShell	Client (Windows)
Desktop	Screenshot, clipboard, processos e janelas	Client (Windows)

1.2 Filosofia do Projeto

"Be genuinely helpful, not performatively helpful. Skip the 'Great question!' and 'I'd be happy to help!' — just help. Actions speak louder than filler words."

- **Modularidade:** cada agente é independente e pode ser substituído ou melhorado isoladamente
- **Segurança:** ferramentas perigosas exigem confirmação; Cloud e Client são isolados
- **Extensibilidade:** novas ferramentas e agentes podem ser adicionados sem alterar o núcleo
- **Transparência:** streaming de eventos permite que o usuário veja o progresso em tempo real

2. Arquitetura Geral

2.1 Diagrama de Alto Nivel

O colabor-ai é composto por **tres modulos principais** que se comunicam via WebSocket e compartilham tipos TypeScript:



2.2 Fluxo de uma Conversa

```
Usuario digita: "organize meus downloads por tipo de arquivo"
|
v
[Client] envia mensagem via WebSocket para o Cloud
|
v
[Cloud] Orchestrator recebe e aciona o PlannerAgent
|
v
[Planner] Decide: "file_system: liste os arquivos de Downloads"
|
v
[Cloud] Envia tool_call para o Client via WebSocket
|
v
[Client] Executa list_dir em C:\Users\...\Downloads
|
v
[Client] Retorna tool_result com a lista de arquivos
|
v
[Cloud] Planner avalia e decide: "file_system: mova .pdf para /PDFs..."
|
v
[Cloud] Envia nova tool_call (write_file / create_dir / move)
|
v
[Client] Executa as operacoes e retorna tool_result
|
v
[Cloud] Planner: "finish" | WriterAgent formata resposta
|
v
[Client] Exibe: "Pronto! 42 arquivos organizados em 3 pastas."
```

3. Modulo Core (Engine)

O **Core** é a biblioteca compartilhada que contém as classes fundamentais usadas por todos os módulos. Está na raiz do projeto (`core/`) e define a base para agentes, orquestradores e ferramentas.

3.1 Agent (`core/agent/agent.ts`)

Cada agente é um wrapper sobre um modelo de linguagem (OpenAI/DeepSeek) que encapsula: **personalidade** (nome, papel, objetivo, backstory), **historico de conversa** (contexto da sessão), **tools** (ferramentas que pode chamar) e **system prompt** (instruções de comportamento).

Loop de Tool Calling: Quando o Agent detecta que o LLM retornou `tool_calls`, ele executa as funções registradas, adiciona os resultados ao histórico, e reenvia para o LLM — repetindo até que o modelo produza uma resposta textual final.

```
const agent = new Agent({
  name: "MathAgent",
  role: "Math expert",
  goal: "Solve mathematical problems",
  backstory: "An AI specialized in calculations.",
  model: "gpt-5-nano",
  tools: [pythonExecTool],
  functions: { execute_python: pythonExecTool.handler }
});
const resposta = await agent.run("Quanto é 352 x 127?");
```

3.2 AgentOrchestrator (`core/orchestrator/orchestrator.ts`)

O **AgentOrchestrator** é o maestro que coordena múltiplos agentes:

```

+-----+
| ORCHESTRATOR LOOP |
| 1. Planner decide o que fazer |
| 2. Planner escolhe um agente |
| 3. Agente executa a tarefa |
| 4. Resultado e registrado |
| 5. Planner decide: continuar |
|    ou "finish"? |
| 6. Se finish -> responde! |
|    Se nao -> volta ao passo 1 |
| |
| (max. 10 iteracoes, protecao |
|    contra loops infinitos) |
+-----+

```

O Planner responde sempre em JSON:

```

// Precisa de acao:
{ "agent": "python_code", "instruction": "Calcule 352 x 127 usando Python" }

// Satisfeito:
{ "agent": "finish", "instruction": "O resultado e 44.704" }

```

3.3 ToolRegistry & Tools

Componente	Descricao
<code>ToolDefinition</code>	Interface com name, description, parameters (JSON Schema) e funcao execute
<code>ToolRegistry</code>	Registro que armazena, busca e converte tools para formato OpenAI
<code>pythonExecTool</code>	Executa codigo Python via child_process com timeout de 10s
<code>getCurrentDateTime</code>	Retorna data/hora atual do sistema em ISO 8601

3.4 Agentes Especializados

Agente	Funcao	Status
chatAgent	Conversa geral e respostas	feito
criticAgent	Avalia se resposta resolveu (SATISFIED/NOT_SATISFIED)	feito
dailyPlannerAgent	Organizacao de tarefas e planejamento diario	feito
executorAgent	Execucao de tarefas de um plano	feito
formatterAgent	Formatacao de respostas	feito
researchAgent	Pesquisa e analise de informacoes	feito
summarizerAgent	Resumo e simplificacao de textos	feito

4. Modulo Cloud (Backend)

O Cloud e o **cerebro** do assistente. Ele pensa, decide e coordena tudo. O app (Client) so mostra o resultado.

4.1 Servidor (Express + WebSocket)

O Cloud roda na porta 3001 com os seguintes endpoints:

Endpoint	Metodo	Descricao
/health	GET	Health check
/auth/login	POST	Login com API key, retorna JWT
/auth/verify	POST	Verifica validade do token
/auth/refresh	POST	Renova token JWT
/chat/message	POST	Envia mensagem, recebe resposta completa
/chat/stream/:id	GET	Info do endpoint WebSocket
/chat/sessions	GET	Lista sessoes ativas
/ws	WebSocket	Conexao em tempo real

4.2 Orchestrator Cloud Edition

Suporta agentes **cloud** (executados no servidor) e **local** (executados no Client via WebSocket):

Tipo	Agentes	Onde executam
Cloud	assistant, python_code, writer	Servidor, via LLM
Local	file_system, shell, desktop	Client Windows, via WebSocket tool_call

4.3 Memory Engine

Sistema de memória de longo prazo com:

- **MEMORY.md:** fatos permanentes que nunca são esquecidos
- **Notas diárias (memory/):** registros automáticos de cada conversa
- **Busca por similaridade:** tokenização + scoring para encontrar memórias relevantes
- **Cache TTL (10s):** evita leitura repetida de arquivos
- **Consolidação:** extrai fatos e decisões importantes das conversas

4.4 Context Engine

Gerencia o orçamento de tokens do LLM com estratégia híbrida:

Parametro	Valor	Descricao
maxTokens	8000	Limite máximo de tokens no contexto
recentRatio	0.6 (60%)	Proporção reservada para mensagens recentes
keepRecentIntact	5	Últimas N mensagens nunca são resumidas
mode	summarize	Usa LLM para resumir em vez de truncar

Exemplo: Numa conversa de 20 mensagens que excede 8000 tokens, o Context Engine mantém as 5 últimas intactas, resume as 15 anteriores em um parágrafo usando o LLM.

4.5 Autenticação JWT

- **Login:** aceita API key ou cria sessão anônima
- **Token JWT:** assinado com HMAC-SHA256, expira em 24h
- **Middleware:** protege todas as rotas /chat/*
- **Refresh:** renova token sem reautenticar
- **WebSocket:** token passado como query param ?token=...

5. Modulo Client (Desktop)

O Client e o **corpo** do assistente. A interface que o usuario ve e as maos que executam acoes reais no Windows.

5.1 Visao Geral do Client

Tecnologia	Funcao
Electron	Empacota o app como executavel nativo (.exe) com acesso ao sistema de arquivos, shell e processos do Windows
React 18	Interface de usuario reativa e componentizada
Vite 5	Build tool e dev server com HMR (Hot Module Replacement)
TypeScript	Tipagem estatica para seguranca e produtividade
WebSocket (ws)	Comunicacao bidirecional em tempo real com o Cloud

5.2 Ferramentas Locais

O Client expoe **13 ferramentas** que o Cloud pode solicitar via WebSocket:

Categoria	Ferramenta	Descricao	Confirmacao
File System	read_file	Le um arquivo do disco	Nao
	write_file	Cria ou sobrescreve arquivo	Sim
	list_dir	Lista conteudo de uma pasta	Nao
	create_dir	Cria uma nova pasta	Nao
	delete_file	Deleta um arquivo	Sim
Shell	run_cmd	Executa comando no CMD	Sim
	run_powershell	Executa script PowerShell	Sim
Desktop	get_env	Le variaveis de ambiente	Nao
	screenshot	Tira print da tela	Sim
	clipboard_get	Le area de transferencia	Nao
	clipboard_set	Escreve na area de transferencia	Nao
	list_processes	Lista programas em execucao	Nao
	list_windows	Lista janelas abertas	Nao

Seguranca: Ferramentas marcadas como **Sim** exigem confirmacao explicita do usuario antes de serem executadas.

6. Protocolo de Comunicacao

6.1 WebSocket Messages

Toda comunicacao Cloud-Client usa JSON sobre WebSocket, definido em `cloud/src/protocol/tool-protocol.ts`.

Cloud -> Client

Tipo	Payload	Quando
connected	sessionId, message, userId	Apos autenticacao WebSocket
tool_call	id, agent, tool, params, requireConfirmation, description	Precisa executar ferramenta local
stream	chunkType (text tool_call progress end error), content, agent	Streaming da resposta
error	message, sessionId	Erro no processamento
pong	timestamp	Resposta ao ping (keepalive)

Client -> Cloud

Tipo	Payload	Quando
chat	message	Usuario envia mensagem
tool_result	id, status (ok error cancelled), result, error	Apos executar ferramenta
ping	(opcional)	Keepalive periodico
confirmation_response	id, approved (boolean)	Resposta ao dialogo de confirmacao

6.2 Fluxo Tool Call/Result



7. Stack Tecnologica

Camada	Core	Cloud	Client
Linguagem	TypeScript	TypeScript	TypeScript
Runtime	Node.js + tsx	Node.js + ts-node	Electron + Vite
Framework	—	Express	React 18
Comunicacao	—	WebSocket (ws)	WebSocket (ws)
LLM	OpenAI SDK	OpenAI SDK + DeepSeek	—
Auth	—	JWT (jsonwebtoken)	—
Testes	Jest + Babel	Jest + ts-jest	—
Linting	git-commit-msg-linter	ESLint + Prettier	ESLint + Prettier
Docs	README.md	TypeDoc + Tailwind	README.md
Build	tsx (on-the-fly)	tsc	Vite + electron-builder

8. Guia de Inicio Rapido

Pre-requisitos

- Node.js 18+
- Python 3.x (para execute_python tool)
- API Key DeepSeek ou OpenAI

Instalacao

```
git clone https://github.com/adm4670/colabor-ai.git
cd colabor-ai
npm install
cd cloud && npm install && cd ..
cd client && npm install && cd ..
```

Configuracao

```
# .env na raiz (Core)
OPENAI_API_KEY=sk-...
TELEGRAM_TOKEN=123456:ABC...

# cloud/.env
DEEPSEEK_API_KEY=sk-...
LLM_PROVIDER=deepseek
JWT_SECRET=seu-segredo
PORT=3001
```

Executando

```
# Core: agente unico no terminal
npm run start:dev:agent

# Core: multi-agente no terminal
npm run start:dev:orchestrator

# Cloud: servidor backend
cd cloud && npm run dev

# Client: app Electron (modo dev)
cd client && npm run dev
```

Testes

```
npm test          # Core
cd cloud && npm test # Cloud (42 testes)
```

9. Roadmap

Concluido

Item	Modulo
Agent unico com tools e historico	Core
Orquestrador multi-agente	Core
Ferramenta execute_python	Core
Canais: Telegram + Terminal	Core
Testes com Jest + cobertura 91%	Core
Servidor Express + WebSocket	Cloud
Autenticacao JWT	Cloud
Memory Engine com busca	Cloud
Context Engine com sumarizacao	Cloud
Tool Protocol (tool_call/tool_result)	Cloud
Scaffold Electron + React + Vite	Client
42 testes automatizados	Cloud

Em Andamento

Item	Status
Stream de eventos (progresso em tempo real)	parcial
Memoria de longo prazo (persistencia)	parcial

Planejado

Item	Prioridade
Implementar src/ (React UI) do Client	alta
Implementar electron/ (ferramentas locais) do Client	alta
Dashboard Web para agentes	media
Multi-usuario com isolamento de sessoes	media
Persistencia em banco de dados	media
Novos canais: WhatsApp, Slack, Discord	baixa
Tool marketplace	baixa
No-code agent builder	baixa
Observabilidade (logs, metricas, tracing)	baixa
Workflow engine com branches	baixa

Nota: Este documento foi gerado a partir da analise do codigo-fonte do repositorio colabor-ai. Para informacoes atualizadas, consulte o repositorio oficial.